



Department of Information and Communication Technology

Faculty of Technology

University of Ruhuna

Database Management Systems Practicum

ICT 1222

Assignment 01 – Mini Project

Group 07

Submitted to: Ms. Sandaruwani Pathirage

Submitted by: TG/2024/2066 – Senitha Samarasinghe

TG/2024/2067 – Dasindu Dilvan

TG/2024/2068 – Maduka Manaram

TG/2024/2107 – Dulsha Hemini

Table of Contents

1. Introduction.....	3
2. Project Objectives	3
3. Scope of the System.....	3
4. Problem Statement.....	4
5. Functional Requirements	4
6. Non-Functional Requirements	4
7. System Users	4
8. Database Design	5
9. Entity Relationship Diagram	5
10. Entity Relationship Description.....	6
11. Tables and Attributes	6
12. Relational Mapping Diagram	7
13. Relationships Between Tables	8
14. SQL Implementation	8
15. Views Used in the System	8
16. User Management and Privileges	10
17. Sample Queries	10
18. Stored Procedures	11
18.1 Check Exam Eligibility Procedure.....	11
18.2 Update Student GPA Procedure	11
18.3 Calculate Final Result Procedure	11
19. Testing and Validation.....	12
20. Challenges Faced	12
21. Individual Contribution	12
22. Conclusion	17
23. Final Outcome	17
24. GitHub Repository.....	17
25. References.....	17

1. Introduction

The Faculty Management System is a database driven system developed for the Faculty of Technology to manage academic and administrative activities efficiently. The system handles student details, lecturer information, course management, student-course enrollments, attendance tracking, medical requests, examination results, and eligibility calculations.

The project was developed as part of the Database Management Systems Practicum module using MySQL and SQL concepts such as tables, relationships, constraints, views, joins, and user privileges.

2. Project Objectives

The main objectives of the project are:

- To manage student information efficiently.
- To maintain lecturers and department records.
- To manage courses and enrollment information.
- To calculate attendance eligibility automatically.
- To manage examination marks and grades.
- To generate useful academic reports using SQL views.
- To implement role-based database security.

3. Scope of the System

The system covers the following academic management activities:

- Student management
- Lecturer management
- Department management
- Course management
- Manage student enrollment
- Attendance recording
- Medical request management
- Examination result management
- Eligibility calculation
- Database user privilege management

4. Problem Statement

Managing faculty activities manually is time consuming and error prone. Attendance calculations, result processing, and eligibility determination become difficult when handled using paper-based systems or spreadsheets.

The Faculty Management System was developed to automate these operations and provide accurate and secure data management.

5. Functional Requirements

The system should be able to:

- Add, update, and delete student records.
- Add lecturer and technical officer records.
- Maintain department information.
- Manage course information.
- Enroll students into courses.
- Record attendance for course sessions.
- Process medical requests.
- Store examination marks.
- Calculate final grades.
- Determine end examination eligibility.
- Generate attendance and result reports.

6. Non-Functional Requirements

- Data consistency
- Database security
- High accuracy
- Scalability
- Reliability
- Easy maintenance
- Multiuser access support

7. System Users

User Type	Responsibilities
Admin	Full database control
Dean	Academic management and monitoring
Lecturer	Manage marks and academic activities
Technical Officer	Attendance and session management
Student	View academic information

8. Database Design

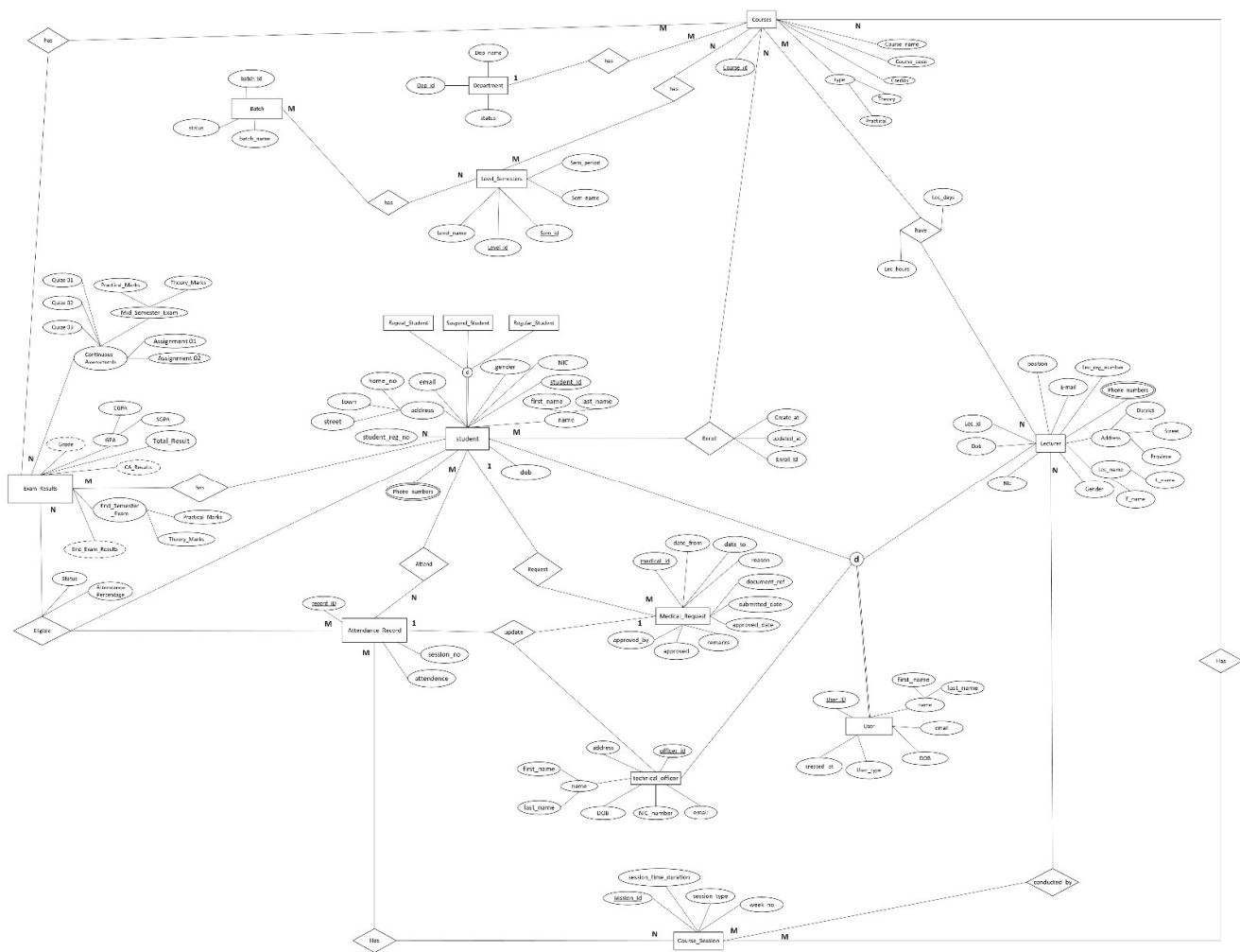
The database was designed using relational database concepts.

Main entities include:

- Department
- Batch
- Student
- Lecturer
- Technical_Officer
- Course
- Enrollment
- Course_Session
- Attendance_Record
- Medical_Request
- Exam_Results
- Eligible

Primary keys and foreign keys were used to maintain relational integrity.

9. Entity Relationship Diagram



10. Entity Relationship Description

Main Relationships

Relationship	Type
Department → Student	One to Many
Department → Lecturer	One to Many
Department → Course	One to Many
Student → Enrollment	One to Many
Course → Enrollment	One to Many

11. Tables and Attributes

Student Table:

Attribute	Data Type	Description
student_id	INT	Primary Key
student_reg_no	VARCHAR	Registration Number
nic	VARCHAR	National Identification No
first_name	VARCHAR	Student first name
last_name	VARCHAR	Student last name
contact_num	VARCHAR	Student Contact No
gender	VARCHAR	Gender
dob	DATE	Date of birth
email	VARCHAR	Student email
Home_num	VARCHAR	Home No of Address
Street	VARCHAR	Street of Address
Town	VARCHAR	Town of Address
Student_type	ENUM	Regular/Repeat/Suspend
sgpa	DECIMAL	Semester GPA
cgpa	DECIMAL	Cumulative GPA
user_id	INT	Foreign_key

Course Table:

Attribute	Data Type	Description
course_id	INT	Primary Key
course_code	VARCHAR	Course code
course_name	VARCHAR	Course name
credits	INT	Course credits
course_type	ENUM	Theory/Practical
department_id	INT	Foreign Key
level_id	INT	Foreign Key
sem_id	INT	Foreign Key

Exam_Results Table:

Attribute	Description
Result_id	Primary Key
Student_id	Foreign Key
Course_id	Foreign Key
quiz01_marks	Quiz 1 marks
quiz02_marks	Quiz 2 marks
quiz03_marks	Quiz 3 marks
assignment01_marks	Assignment 1 marks
assignment02_marks	Assignment 2 marks
mid_exam_theory_marks	Mid theory marks
mid_exam_practical_marks	Mid practical marks
ca_result	Continuous assessment result
end_exam_theory_marks	End theory marks
end_exam_practical_marks	End practical marks
end_exam_result	End examination result
total_result	Final result
grade	Final grade

12. Relational Mapping Diagram



13. Relationships Between Tables

The database uses foreign key relationships to ensure consistency.

```
FOREIGN KEY (student_id) REFERENCES Student(student_id) ON DELETE CASCADE,  
FOREIGN KEY (course_id) REFERENCES Course(course_id) ON DELETE CASCADE
```

14. SQL Implementation

Sample Insert Query

```
mysql> INSERT INTO Eligible (student_id, course_id, attendance_percentage, status) VALUES  
-> (21, 11, 88, 'Eligible');  
Query OK, 1 row affected (0.05 sec)
```

Sample User Creation

```
mysql> CREATE USER 'admin'@'localhost'  
-> IDENTIFIED BY 'Admin@123';  
Query OK, 0 rows affected (0.01 sec)
```

Grant Privileges

```
mysql> GRANT ALL PRIVILEGES  
-> ON Faculty_Management_System.*  
-> TO 'admin'@'localhost'  
-> WITH GRANT OPTION;  
Query OK, 0 rows affected (0.01 sec)
```

15. Views Used in the System

End_Exam_Eligibility View

This view displays students who are eligible for end examinations based on attendance percentage.

```
mysql> CREATE VIEW End_Exam_Eligibility AS  
-> SELECT  
->     s.student_id,  
->     s.student_reg_no,  
->     CONCAT(s.first_name, ' ', s.last_name) AS student_name,  
->     c.course_code,  
->     c.course_name,  
->     e.attendance_percentage,  
->     e.status  
-> FROM Eligible e  
-> JOIN Student s  
->     ON e.student_id = s.student_id  
-> JOIN Course c  
->     ON e.course_id = c.course_id  
-> WHERE e.attendance_percentage >= 80;  
Query OK, 0 rows affected (0.02 sec)
```

End_Exam_Results View

This view shows student examination results.

```
mysql> CREATE VIEW End_Exam_Results AS
-> SELECT
->     s.student_id,
->     s.student_reg_no,
->     CONCAT(s.first_name, ' ', s.last_name) AS student_name,
->     c.course_code,
->     c.course_name,
->     er.ca_result,
->     er.end_exam_result,
->     er.total_result,
->     er.grade
-> FROM Exam_Results er
-> JOIN Student s
->     ON er.student_id = s.student_id
-> JOIN Course c
->     ON er.course_id = c.course_id;
Query OK, 0 rows affected (0.01 sec)
```

Student_Attendance View

This view calculates attendance percentages.

```
mysql> CREATE VIEW Student_Attendance AS
-> SELECT s.student_id, s.student_reg_no,
->     CONCAT(s.first_name, ' ', s.last_name) AS student_name,
->     c.course_code,
->     c.course_name,
->     COUNT(ar.record_id) AS total_sessions,
->     SUM(
->         CASE
->             WHEN ar.attendance = 'Present'
->             THEN 1
->             ELSE 0
->         END
->     ) AS present_count,
->     ROUND(
->         (
->             SUM(
->                 CASE
->                     WHEN ar.attendance = 'Present'
->                     THEN 1
->                     ELSE 0
->                 END
->             ) / COUNT(ar.record_id)
->         ) * 100,
->         2
->     ) AS attendance_percentage
-> FROM Attendance_Record ar
-> JOIN Student s ON ar.student_id = s.student_id
-> JOIN Course_Session cs ON ar.session_id = cs.session_id
-> JOIN Enrollment e ON s.student_id = e.student_id
-> JOIN Course c ON e.course_id = c.course_id
-> GROUP BY s.student_id, c.course_id;
Query OK, 0 rows affected (0.03 sec)
```

16. User Management and Privileges

Different privilege levels were assigned for security purposes.

User	Privileges
Admin	Full access
Dean	Full database access
Lecturer	Full database access
Technical Officer	Attendance and session management
Student	Limited access

17. Sample Queries

Retrieve Eligible Students

```
mysql> SELECT *
-> FROM End_Exam_Eligibility;
```

student_id	student_reg_no	student_name	course_code	course_name	attendance_percentage	status
1	ICT/2020/001	Amali Perera	ICT1112	Essentials of ICT and Computer Hardware	85	Eligible
1	ICT/2020/001	Amali Perera	ICT1153	Web Technologies	92	Eligible
1	ICT/2020/001	Amali Perera	ICT1233	Server-Side Web Development	80	Eligible
2	ICT/2020/002	Nuwan Bandara	ICT1132	Fundamentals of Computer Programming	88	Eligible
2	ICT/2020/002	Nuwan Bandara	ICT1153	Web Technologies	95	Eligible
2	ICT/2020/002	Nuwan Bandara	ICT2122	Object Oriented Programming	82	Eligible

Retrieve Student Results

```
mysql> SELECT *
-> FROM End_Exam_Results;
```

student_id	student_reg_no	student_name	course_code	course_name	ca_result	end_exam_result	total_result	grade
1	ICT/2020/001	Amali Perera	ICT1112	Essentials of ICT and Computer Hardware	80.00	85.00	82.50	A
1	ICT/2020/001	Amali Perera	ICT1132	Fundamentals of Computer Programming	72.50	78.00	75.30	B+
1	ICT/2020/001	Amali Perera	ICT1153	Web Technologies	91.00	86.50	88.80	A
1	ICT/2020/001	Amali Perera	ICT1212	Database Management Systems	63.00	65.00	64.00	B
1	ICT/2020/001	Amali Perera	ICT1233	Server-Side Web Development	74.50	80.00	77.30	B+
1	ICT/2020/001	Amali Perera	ICT2113	Data Structures and Algorithms	70.00	73.50	71.80	B
2	ICT/2020/002	Nuwan Bandara	ICT1112	Essentials of ICT and Computer Hardware	67.00	70.00	68.50	B
2	ICT/2020/002	Nuwan Bandara	ICT1132	Fundamentals of Computer Programming	57.50	60.00	58.80	C+
2	ICT/2020/002	Nuwan Bandara	ICT1153	Web Technologies	82.50	81.00	81.80	A-
2	ICT/2020/002	Nuwan Bandara	ICT1212	Database Management Systems	72.00	75.00	73.50	B+
2	ICT/2020/002	Nuwan Bandara	ICT1233	Server-Side Web Development	53.00	55.00	54.00	C
2	ICT/2020/002	Nuwan Bandara	ICT2122	Object Oriented Programming	79.00	79.00	79.00	B+

Retrieve Attendance Information

```
mysql> SELECT *
-> FROM Student_Attendance;
```

student_id	student_reg_no	student_name	course_code	course_name	total_sessions	present_count	attendance_percentage
1	ICT/2020/001	Amali Perera	ICT1112	Essentials of ICT and Computer Hardware	10	8	80.00
1	ICT/2020/001	Amali Perera	ICT1132	Fundamentals of Computer Programming	10	8	80.00
1	ICT/2020/001	Amali Perera	ICT1153	Web Technologies	10	8	80.00
1	ICT/2020/001	Amali Perera	ICT1212	Database Management Systems	10	8	80.00
1	ICT/2020/001	Amali Perera	ICT1233	Server-Side Web Development	10	8	80.00
1	ICT/2020/001	Amali Perera	ICT2113	Data Structures and Algorithms	10	8	80.00
5	ICT/2020/005	Kasun Liyanage	ICT1112	Essentials of ICT and Computer Hardware	7	3	42.86
5	ICT/2020/005	Kasun Liyanage	ICT1132	Fundamentals of Computer Programming	7	3	42.86
5	ICT/2020/005	Kasun Liyanage	ICT1153	Web Technologies	7	3	42.86
5	ICT/2020/005	Kasun Liyanage	ICT1212	Database Management Systems	7	3	42.86
5	ICT/2020/005	Kasun Liyanage	ICT2113	Data Structures and Algorithms	7	3	42.86
5	ICT/2020/005	Kasun Liyanage	ICT4133	Artificial Intelligence	7	3	42.86

18. Stored Procedures

18.1 Check Exam Eligibility Procedure

The Eligibility_Check procedure is used to determine whether a student is eligible to sit for the final examination based on attendance percentage. This procedure takes: Student ID and Course ID.

First, it calculates the total number of sessions conducted for the selected course. Then it counts the number of sessions attended by the student where the attendance status is marked as "Present".

If the attendance percentage is 80% or higher, the student is marked as "Eligible" in the Eligible table. Otherwise, the student is marked as "Not Eligible".

This procedure helps to:

- Reduce manual attendance calculations
- Improve accuracy
- Automatically update eligibility records

18.2 Update Student GPA Procedure

The GPA_Calculation procedure is used to calculate and update the SGPA and CGPA of students.

This procedure takes: Student ID

The procedure retrieves grades from the Exam_Results table and course credits from the Course table. Each grade is converted into grade points using a grading scale.

Finally, the calculated SGPA and CGPA values are updated in the Student table.

This procedure helps to:

- Automate GPA calculations
- Reduce human errors
- Maintain accurate academic records

18.3 Calculate Final Result Procedure

The Result_Calculation procedure is used to calculate the final marks and grades of students.

This procedure takes:

- Result ID

The procedure calculates:

- Quiz average
- Continuous Assessment (CA) marks
- End examination results
- Final total marks

The procedure updates the Exam_Results table with:

- CA results
- End exam results
- Total marks
- Final grades

This procedure helps to:

- Automate result processing
- Improve consistency
- Reduce manual calculations
- Speed up examination evaluation

19. Testing and Validation

The database was tested using:

- Insert queries
- Update queries
- Delete queries
- Join operations
- View generation
- User privilege testing

Test data for 60 students across three departments was inserted successfully.

20. Challenges Faced

- Managing large datasets
- Designing relationships correctly
- Avoiding duplicate records
- Calculating attendance percentages accurately
- Managing SQL syntax errors in large scripts

21. Individual Contribution

Member Index	Member Name	Project Module
TG/2024/2066	Senitha Samarsinghe	Marks, Grades & Results
TG/2024/2067	Dasindu Dilvan	Attendance Management
TG/2024/2068	Maduka Manaram	User & Role Management
TG/2024/2107	Dulsha Hemini	Course Unit Management

Senitha Samarasinghe

TG/2024/2066



Marks, Grades & Results Module

Responsibilities

I was responsible for designing and implementing the examination, grading, and result processing component of the Faculty Management System.

Main Tasks Performed

- Designed the Exam_Results table structure.
- Managed examination components including:
 - Quiz marks
 - Assignment marks
 - Mid theory exams
 - Mid practical exams
 - Continuous assessment (CA)
 - Final theory exams
 - Final practical exams
- Calculated with Result_Calculation stored procedure:
 - CA results
 - End exam results
 - Final total results
 - Grades
- Inserted large-scale realistic sample datasets for testing.
- Implemented grading logic using university grading standards.
- Created result reporting views.
- Managed academic performance analysis.

SQL Features Implemented

- Large-scale INSERT INTO operations
- Numeric calculations
- Grade processing logic
- Result aggregation
- CREATE VIEW End_Exam_Results
- Performance reporting queries
- Result_Calculation stored procedure

Contribution to the System

This module automated examination result processing and reduced the complexity of manual grading systems. It improved accuracy, efficiency, and accessibility of student academic records.

Dasindu Dilvan

TG/2024/2067



Attendance Management Module

Responsibilities

I developed the attendance management and eligibility tracking component of the system. This module monitors student attendance and determines eligibility for examinations based on attendance percentages.

Main Tasks Performed

- Designed attendance-related database tables.
- Implemented attendance recording for each course session.
- Managed attendance status such as:
 - Present
 - Absent
- Developed session management functionalities.
- Created attendance percentage calculation mechanisms.
- Implemented eligibility determination logic for end examinations.
- Inserted attendance and eligibility sample data.
- Developed attendance analytical views and reports.
- Developed stored procedure for check end exam Eligibility

SQL Features Implemented

- Aggregate functions:
 - COUNT()
 - SUM()
 - ROUND()
- Conditional calculations using CASE
- GROUP BY
- Attendance percentage calculations
- CREATE VIEW Student_Attendance
- CREATE VIEW End_Exam_Eligibility
- Eligibility_Check Stored Procedure

Contribution to the System

This module automated attendance tracking and reduced manual errors in eligibility calculations. It helped lecturers and administrators monitor student participation effectively and ensured fair examination eligibility processing.

Maduka Manaram

TG/2024/2068



User & Role Management Module

Responsibilities

I was responsible for designing and implementing the complete user and access management component of the Faculty Management System. This module ensures that different categories of users can securely access the system according to their responsibilities and permissions.

Main Tasks Performed

- Designed the database structure for storing user-related information.
- Implemented role-based access control for:
 - Admin
 - Dean
 - Lecturer
 - Technical Officer
 - Student
- Created MySQL database users using CREATE USER.
- Assigned appropriate database permissions using GRANT PRIVILEGES.
- Managed authentication and authorization mechanisms.
- Ensured system security by restricting unauthorized access.
- Supported secure database operations for multiple user types.

SQL Features Implemented

- CREATE USER
- GRANT ALL PRIVILEGES
- GRANT SELECT, INSERT, UPDATE
- Role-based privilege allocation

Contribution to the System

This module improved the security and reliability of the system by ensuring that each user can only access functionalities relevant to their role. It also helped maintain data confidentiality and integrity within the faculty management environment.

Dulsha Hemini

TG/2024/2107



Course Unit Management Module

Responsibilities

I handled the design and implementation of the course and enrollment management component. This module manages course units, student enrollments, and academic relationships between departments and courses.

Main Tasks Performed

- Designed course-related database tables.
- Managed course information including:
 - Course ID
 - Course Code
 - Course Name
 - Credit values
 - Semester details
- Implemented student enrollment functionality.
- Created relationships between:
 - Students and Courses
 - Departments and Courses
- Developed SQL queries for course allocation and retrieval.
- Managed course-session relationships.
- Implemented student course registration processes.
- Created the Student_Courses view for retrieving enrolled course information.
- Developed stored procedure for GPA calculation

SQL Features Implemented

- CREATE TABLE
- FOREIGN KEY
- JOIN
- CREATE VIEW
- Enrollment relationship queries
- GPA_Calculation stored procedure

Contribution to the System

This module enabled efficient management of academic course structures and student registrations. It ensured proper organization of course data and supported smooth academic operations within the faculty.

22. Conclusion

The Faculty Management System successfully demonstrates the practical implementation of database concepts such as normalization, relational modeling, SQL queries, views, constraints, joins, and privilege management.

The system improves efficiency, reduces manual work, and provides accurate academic management functionalities for the Faculty of Technology.

23. Final Outcome

The project demonstrates:

- Relational database design
- SQL implementation
- User privilege management
- Attendance processing
- Result processing
- Academic reporting using views
- Realistic dataset generation for testing

The project is suitable as a complete DBMS mini project submission for university level coursework.

24. GitHub Repository

Link: - https://github.com/DulshaHemini/Faculty_Management_System.git

25. References

- Lecture notes and practical materials provided by the Lecturer
- Database Management Systems – [Neso Academy](#)
- W3Schools MySQL Tutorials – [w3schools.com](https://www.w3schools.com)
- AI Debugging Support – [Claude AI](#)